

COS30019 – Introduction to Artificial Intelligence

Assignment 2B Report - Machine Learning and Software Integration

Tutor: Hy Nguyen

Class: Tuesday - 8:30

Group Member	Student ID	Signature
Nicholas Kan	105923474	Nicholas
Charitha Bandara	104178396	Charith
Xinbo Yu	105961513	Xinbo
Santiago Merchant	105342228	Santiago

Contents

1. Instructions	2
2. Introduction	3
3. Features / Bugs / Missing.....	3
3.1 Implemented Features.....	3
3.2 Known Limitations and Missing Items	5
4. Testing.....	5
4.1 Model Testing	5
4.2 Route Guidance Test.....	8
4.3 Software Test Cases	8
5. Insights	9
6. Research.....	10
7. Conclusion	10
8. Acknowledgements / Resources.....	10
9. References.....	11
10. Contribution Statement	12

1. Instructions

The system is operated through Python command-line scripts. The required libraries are under requirements.txt and were trained and tested on a python virtual environment. The workflow is: preprocess the SCATS data, train the models, evaluate model performance, and run the final route guidance system.

The command line inputs are also commented on top of the code files.

Script	Purpose	Output
preprocess_scats.py	Converts raw SCATS V00-V95 columns into timestamped 15-minute records.	processed_scats_15min.csv
train_all_sites.py	Trains MLP, LSTM, and GRU for all SCATS sites.	120 saved .keras models
evaluate_all_sites.py	Compares trained models across all sites.	model comparison CSV files
tbrgs.py	Runs route guidance using origin, destination, time, model, and k routes.	route and edge-detail CSV files

The final route guidance command uses the origin SCATS site, destination SCATS site, date/time, selected model, and number of routes. The time should be within the October 2006 dataset and use 15-minute intervals such as 08:00 or 17:30. Road connections are read from data/scats_connections.csv.

The screenshot displays the Visual Studio Code interface for the 'Assignment2B-Tutor-Style' project. The Explorer panel on the left shows the project structure, including files like __pycache__, data, images, model, results, tests, tests, venv, .gitignore, evaluate_all_sites.py, main.py, preprocess_scats.py, requirements.txt, search_algorithms.py, tbrgs.py, train_all_sites.py, train.py, and traveltime.py. The main editor window shows the content of tbrgs.py, which serves as the main controller for the route guidance system. It lists the components it connects: processed SCATS traffic data, manually created road graph, trained ML models, travel-time conversion functions, and uniform-cost search. Command examples are provided for running the script with different origins, destinations, datetimes, models, and the number of routes (k). An important graph rule is noted: travel is only allowed between SCATS sites listed in data/scats_connections.csv. The bottom panel shows the terminal output of the command: `python tbrgs.py --origin 2000 --destination 2820 --datetime "2006-10-15 08:00" --model lstm --k 5`. The output lists five routes with their travel times and saves the results to results/tbrgs_routes.csv.

```

1  """
2  Traffic-Based Route Guidance System (TBRGS)
3  COS30019 Introduction to AI - Assignment 2B
4
5  This file is the main controller for the route guidance system.
6
7  It connects together:
8  1. data/processed_scats_15min.csv      -> processed SCATS traffic data
9  2. data/scats_connections.csv         -> manually created road graph
10 3. model/created_site_(site).keras    -> trained ML models for each SCATS site
114. traveltime.py                      -> travel-time conversion functions
125. Uniform-cost search                -> returns top-k lowest-time routes
13
14 Command examples:
15 python tbrgs.py --origin 2000 --destination 3002 --datetime "2006-10-15 08:00" --model mlp
16
17 python tbrgs.py --origin 2000 --destination 3002 --datetime "2006-10-15 08:00" --model lstm --k 5
18
19 python tbrgs.py --origin 970 --destination 3685 --datetime "2006-10-20 17:30" --model gru
20
21
22 IMPORTANT GRAPH RULE:
23 -----
24 The program only allows travel between SCATS sites listed in:
25
26 data/scats_connections.csv
27 This file is a manually created list of valid road connections between SCATS sites.
28 The TBRGS program will only consider routes that follow these connections.
29
30 """

```

```

(venv) PS C:\Users\charl\OneDrive\Uni-Sem1-2026\Introduction to AI\Assignment2B-Tutor-Style> python tbrgs.py --origin 2000 --destination 2820 --datetime "2006-10-15 08:00" --model lstm --k 5
Routes:
Route 1: 2000 -> 2280 -> 2820 | Travel time: 11.36 minutes
Route 2: 2000 -> 3685 -> 3126 -> 2820 | Travel time: 11.89 minutes
Route 3: 2000 -> 3126 -> 4060 -> 4835 -> 2820 | Travel time: 12.91 minutes
Route 4: 2000 -> 3685 -> 3126 -> 2280 -> 2820 | Travel time: 13.49 minutes
Route 5: 2000 -> 2280 -> 3126 -> 2820 | Travel time: 13.73 minutes

Results saved to: results/tbrgs_routes.csv
(venv) PS C:\Users\charl\OneDrive\Uni-Sem1-2026\Introduction to AI\Assignment2B-Tutor-Style> python tbrgs.py --origin 2000 --destination 2820 --datetime "2006-10-15 08:00" --model mlp --k 5

```

Figure 1. Example project structure and command-line execution.

2. Introduction

This project developed a Traffic-Based Route Guidance System (TBRGS) for the Boroondara area using SCATS traffic flow data. The system predicts traffic flow at SCATS intersections and uses these predictions to estimate travel time and recommend up to five routes between an origin and destination SCATS site.

The dataset contains traffic flow readings every 15 minutes for October 2006. The raw data was preprocessed into a time-series format containing the SCATS site number, location, latitude, longitude, timestamp, and flow value. Three prediction models were implemented: MLP, LSTM, and GRU. One of each model type was trained for every SCATS site, producing 120 models in total.

The route guidance component integrates Assignment 2A with Assignment 2B. In Assignment 2A, search algorithms found paths through graph nodes using edge costs. In this system, nodes are SCATS sites, edges are road connections, and edge costs are dynamically calculated travel times. The adapted CUS1 / Uniform Cost Search algorithm was used because it expands routes by the lowest accumulated travel time.

For each edge A to B, the system predicts traffic flow at site B, converts the 15-minute flow into hourly flow, estimates travel time using distance and speed assumptions, and adds a 30-second intersection delay. This creates an end-to-end AI prototype that combines data processing, ML prediction, travel-time estimation, and graph search.

3. Features / Bugs / Missing

3.1 Implemented Features

Component	Implemented Feature
Data processing	Raw SCATS data was reshaped into timestamped 15-minute flow records.
ML models	MLP, LSTM, and GRU were trained for all 40 SCATS sites.
Evaluation	Models were compared using MAE, MSE, RMSE, MAPE, R2, and explained variance.
Travel-time cost	Predicted flow, distance, speed limit, and intersection delay were used to calculate edge cost.
Part A integration	CUS1 / Uniform Cost Search was adapted to find top-k lowest-time routes.
Output	The system prints top routes and saves route summary and edge-detail CSV files.

Data Processing

The data processing component converts the raw SCATS traffic dataset into a format suitable for machine learning. The original dataset contains traffic flow values across 96 columns, where each column represents a 15-minute interval in a day. The preprocessing script reshapes this wide format into a long time-series format with one row per SCATS site, timestamp, and traffic flow value.

The processed dataset includes the SCATS site number, location, latitude, longitude, datetime, and flow value. This cleaned dataset is then used to create training sequences, where the previous 12 traffic flow values are used to predict the next 15-minute traffic flow. This allows the models to learn short-term traffic patterns from historical SCATS data.

ML Models

Three machine learning models were implemented for traffic flow prediction: MLP, LSTM, and GRU. The MLP model was used as the additional selected model, while LSTM and GRU were included as the required deep learning models. Each model was trained separately for every SCATS site so that the traffic pattern of each intersection could be learned individually.

The system trained 120 models in total, consisting of one MLP, one LSTM, and one GRU model for each of the 40 SCATS sites. The trained models were saved as .keras files and later loaded by the TBRGS when traffic flow predictions were needed for route calculation.

Evaluation

The evaluation component compares the performance of the trained MLP, LSTM, and GRU models. Each trained model was tested using the test portion of the SCATS time-series data. The main metrics used were MAE, MSE, RMSE, MAPE, R^2 score, and explained variance.

The results showed that GRU performed best overall based on average RMSE, with LSTM performing very closely behind it. MLP was useful as a baseline model but had a slightly higher RMSE. The evaluation results were saved into CSV files so they could be used for comparison, reporting, and deciding which model was most suitable for traffic prediction.

Travel-Time Cost

The travel-time cost component converts predicted traffic flow into an edge cost for route searching. For each road edge from SCATS site A to SCATS site B, the system uses the predicted traffic flow at site B. Since the model predicts a 15-minute flow value, this is converted into an estimated hourly flow before being used in the travel-time calculation.

The system estimates travel time using the distance between the two SCATS sites, the predicted traffic flow, the estimated speed, and a fixed 30-second delay for passing through a controlled intersection. This calculated travel time becomes the edge cost used by the route-search algorithm.

Part A Integration

Assignment 2A was integrated into the TBRGS through the route-search component. In Assignment 2A, the team implemented graph search algorithms that searched for paths using fixed edge costs. In Assignment 2B, this idea was adapted so that SCATS sites became the graph nodes and road connections became the graph edges.

The final TBRGS uses an adapted CUS1 / Uniform Cost Search approach because the goal is to find the lowest total travel time. Instead of using fixed edge costs, the search algorithm receives dynamic travel-time costs calculated from the ML traffic predictions. This connects the search work from Assignment 2A with the machine learning work from Assignment 2B.

Output

The final TBRGS outputs up to five recommended routes between a selected origin and destination SCATS site. Each route shows the sequence of SCATS site numbers and the estimated total travel time in minutes. This makes the output easy to read from the terminal during testing.

The system also saves the route results into CSV files. One file stores the route summary, including the route number, path, and total travel time. Another file stores detailed edge information such as distance, predicted flow, estimated speed, and edge travel time. These outputs were used as testing evidence and report results.

3.2 Known Limitations and Missing Items

Item	Description
Approximate SCATS graph	The dataset did not provide a verified road network, so connections were manually or approximately created.
Straight-line distance	Latitude and longitude were used to estimate distance instead of exact road distance.
Dataset range	Predictions are most reliable within the October 2006 dataset.
GUI	A full GUI was not completed; the current prototype is command-line based.
Model loading	Models load during route calculation; preloading would improve efficiency.

Overall, the main required components of the TBRGS were implemented. The system can preprocess SCATS traffic data, train three different model types, evaluate the trained models, estimate travel time, and return top five route recommendations using a search algorithm adapted from Assignment 2A. The main limitation is that the SCATS road graph is approximate and would require further verification for real-world use.

4. Testing

Testing checked both the machine learning models and the complete TBRGS workflow. The main aim was to confirm that the system could train models, evaluate them, estimate travel time, and return valid route recommendations.

4.1 Model Testing

Item	Result
SCATS sites	40
Model types	MLP, LSTM, GRU
Total trained models	120
Main evaluation outputs	model_comparison_all_sites.csv, model_average_comparison.csv, best_model_by_site.csv

COS30019 Assignment 2B - TBRGS

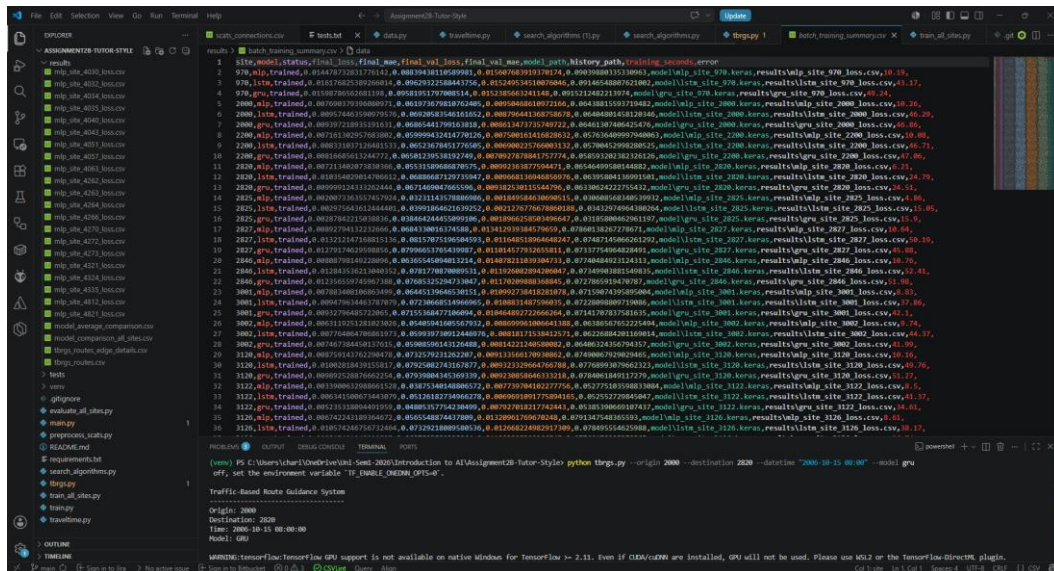


Figure 2. Model training output evidence.

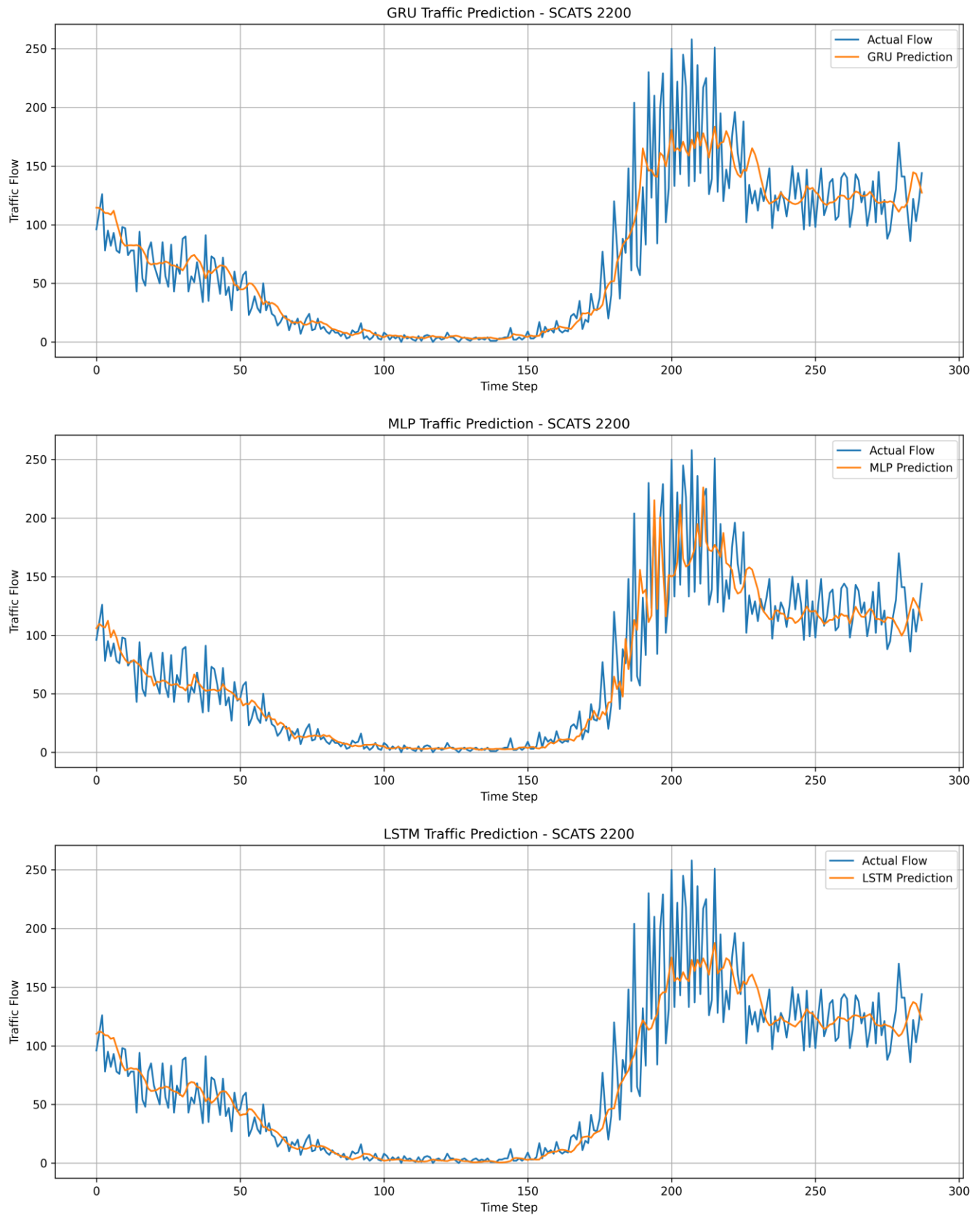


Figure 3. GRU, MLP, and LSTM prediction result line charts in comparison to the actual flow

This figure compares the prediction results for **LSTM, GRU, and MLP** at **SCATS site 2200**. In each graph, the blue line shows the actual traffic flow, while the orange line shows the model prediction.

Comparison graphs are created using the command line: *python main.py --model <model name> --site <site number>* and visual comparison was saved under images.

Overall, all three models follow the main traffic pattern reasonably well. They capture the low-flow period in the middle of the graph and the major increase in traffic flow around time step 180–220. This shows that the models learned the general trend of the traffic data.

The **LSTM and GRU models** produce smoother predictions. They follow the overall shape of the actual flow but do not fully capture the sharp spikes during the peak traffic period. The **MLP model** reacts more sharply to changes and follows some peak movements more closely, but it also appears more unstable during the high-flow region.

This graph supports the numerical evaluation results. GRU and LSTM performed strongly overall because they captured the time-series trend consistently, while MLP was useful as a baseline but produced more variation during peak flow.

4.2 Route Guidance Test

The main route test used origin 2000, destination 2820, time 2006-10-15 08:00, MLP model, and $k = 5$. The system successfully returned five routes.

```
Routes:
Route 1: 2000 -> 3682 -> 3126 -> 2200 -> 4057 -> 4051 -> 4030 -> 4321 -> 4335 -> 2820 | Travel time: 16.65 minutes
Route 2: 2000 -> 3682 -> 3126 -> 2200 -> 4057 -> 4051 -> 4032 -> 4321 -> 4335 -> 2820 | Travel time: 16.94 minutes
Route 3: 2000 -> 3682 -> 3126 -> 3127 -> 3122 -> 4035 -> 4034 -> 4032 -> 4321 -> 4335 -> 2820 | Travel time: 17.13 minutes
Route 4: 2000 -> 3682 -> 3126 -> 3127 -> 3122 -> 3120 -> 4034 -> 4032 -> 4321 -> 4335 -> 2820 | Travel time: 17.46 minutes
Route 5: 2000 -> 3682 -> 3126 -> 2200 -> 4057 -> 4051 -> 4030 -> 4032 -> 4321 -> 4335 -> 2820 | Travel time: 17.75 minutes

Results saved to: results/tbrgs_routes.csv
```

Figure 4. TBRGS top five route outputs.

This test confirmed that the TBRGS could combine the SCATS graph, trained model predictions, travel-time conversion, and adapted CUS1 search to produce multiple route options.

4.3 Software Test Cases

Manual test cases were created to test the TBRGS program performance. Details of the tests were saved as a text file with command line input on the working directory under the tests folder.

No.	Test Area	Input Summary	Expected Result
1	Valid route	2000 to 2820, MLP	Returns up to 5 routes
2	Model variation	LSTM	Uses LSTM predictions
3	Model variation	GRU	Uses GRU predictions
4	Morning traffic	08:00	Valid route output
5	Afternoon traffic	17:30	Valid route output
6	Midday traffic	12:15	Valid route output
7	Date variation	Different October 2006 date	Valid route output
8	Top-k	$k = 5$	Returns up to 5 routes

9	Reverse route	2820 to 2000	Returns reverse route if edges exist
10	Different model/time	GRU with evening time	Valid route output
11	Invalid site	Origin 9999	Rejects invalid site
12	Disconnected site	Unconnected destination	No route found
13	Date outside data	2026 date	Rejects or warns
14	Invalid interval	08:07	Rejects or warns
15	Insufficient history	Very early dataset time	Not enough previous data

These tests cover normal operation, model variation, time variation, invalid input, and graph-related failure cases. The successful route tests confirm Part A and Part B integration, while the error cases check system robustness.

The testing showed that the main functions of the system worked successfully. The program was able to preprocess the dataset, train 120 models, evaluate the models, calculate travel-time edge costs, and return top five route recommendations. The main route test successfully returned five different routes from SCATS site 2000 to SCATS site 2820.

5. Insights

The average model results across all 40 SCATS sites are shown below. RMSE was used as the main comparison metric because it reports prediction errors in traffic-flow units and penalizes larger errors.

```
(venv) PS C:\Users\chari\OneDrive\Uni-Sem1-2026\Introduction to AI\Assignment2B-Tutor-Style> python evaluate_all_sites.py
Average model comparison:
model sites_evaluated average_MAE average_MSE average_RMSE average_MAPE_percent average_R2 average_Explained_Variance
GRU 40 28.601103 1887.804056 40.844592 72.188437 0.727185 0.729545
LSTM 40 28.837016 1897.345248 40.954324 71.307750 0.726134 0.727715
MLP 40 28.722968 2030.261087 42.117678 68.359321 0.707832 0.712219

Best model count by SCATS site:
model
GRU 24
LSTM 8
MLP 8

Evaluation finished.
Saved full comparison to: results/model_comparison_all_sites.csv
Saved average comparison to: results/model_average_comparison.csv
Saved best model by site to: results/best_model_by_site.csv
```

Figure 6. Average RMSE comparison between ML models.

Based on average RMSE, the GRU model performed the best overall. It achieved the lowest average RMSE of 40.84, which means it had the lowest average prediction error magnitude across the 40 SCATS sites. The LSTM model performed very similarly, with an average RMSE of 40.95. This shows that both recurrent models were suitable for traffic flow prediction.

The MLP model had a slightly higher RMSE of 42.12 and a lower R^2 score of 0.708. This suggests that although MLP was able to learn general traffic patterns, it was not as consistent as GRU or LSTM when

predicting time-series traffic flow. This is expected because MLP treats the previous flow values as normal input features, while LSTM and GRU are designed to learn sequential patterns in time-series data.

The GRU and LSTM models performed very closely. The difference between their average RMSE values was small. This small difference shows that both recurrent models captured the traffic flow patterns well. However, GRU slightly outperformed LSTM in this project. One possible reason is that GRU has a simpler structure than LSTM, which can make it easier to train on smaller datasets. Since the provided dataset only covers October 2006, the simpler GRU structure may have been sufficient to learn the traffic flow patterns without needing the additional complexity of LSTM.

6. Research

The main research effort was connecting traffic prediction with graph-based route guidance. The SCATS dataset provided traffic flow and coordinates, but not a verified road network. Therefore, a SCATS connection CSV was created so each SCATS site could act as a node, and each road connection could act as an edge.

A practical travel-time method was implemented using predicted traffic flow, estimated distance, and a 30-second intersection delay. Since exact road distances were not provided, latitude and longitude were used to estimate straight-line distance. Future work should use verified map data, such as OpenStreetMap or VicRoads Road data, to improve road-link accuracy and distance estimation. Preloading models would also improve runtime efficiency.

7. Conclusion

This project successfully developed a prototype Traffic-Based Route Guidance System using SCATS traffic flow data. The system preprocesses the dataset, trains MLP, LSTM, and GRU models, evaluates model performance, estimates edge travel time, and returns up to five route recommendations.

GRU was the best overall model based on average RMSE, while LSTM performed very similarly. MLP was useful as a baseline but was slightly weaker than the recurrent models. The system also successfully integrated Assignment 2A and Assignment 2B by adapting CUS1 / Uniform Cost Search to use dynamic ML-based travel-time edge costs. The main limitation is the approximate SCATS connection graph, which should be improved using verified road network data in future work.

8. Acknowledgements / Resources

Resource	Use
Assignment specification	Defined ML, route guidance, testing, and report requirements.
SCATS dataset	Provided traffic flow data for preprocessing, training, evaluation, and routing.
Python, Pandas, NumPy	Used for data processing and numerical operations.
TensorFlow / Keras	Used to build, train, save, and load MLP, LSTM, and GRU models.
Scikit-learn	Used for scaling and evaluation metrics.
Matplotlib	Used to create evaluation graphs.

Assignment 2A search code	Used as the basis for adapting CUS1 / Uniform Cost Search.
ChatGPT/Copilot	Used for debugging support, code organisation, explanation, and comments formatting

The screenshot shows a VS Code editor with a Jupyter Notebook. The notebook contains a table of results for the TBRGS assignment. The table has the following columns: route_number, edge_number, from_site, to_site, distance_km, predicted_15min_flow_at_to_site, predicted_hourly_flow_at_to_site, estimated_speed_kmh, edge_time_minutes. The results are sorted by route_number and edge_number. The bottom of the notebook shows the command 'Save best model by site: results/best_model_by_site.csv'.

Figure 7. Final testing evidence included in the report.

9. References

Keras. (n.d.). Keras documentation. <https://keras.io/>

NumPy Developers. (n.d.). NumPy documentation. <https://numpy.org/doc/>

Pandas Development Team. (n.d.). Pandas documentation. <https://pandas.pydata.org/docs/>

Scikit-learn Developers. (n.d.). Scikit-learn documentation. <https://scikit-learn.org/stable/>

TensorFlow. (n.d.). TensorFlow documentation. <https://www.tensorflow.org/>

VicRoads. (2006). SCATS traffic flow dataset for Boroondara. Provided through COS30019 Assignment 2.

Swinburne University of Technology. (2026). COS30019 Introduction to AI Assignment 2 specification.

Provided example repository : <https://github.com/hyswin/Example-LSTM-and-GRU-for-A2B-COS30019-.git>

10. Contribution Statement

Team Member	Student ID	Main Contribution Area	Contribution Percentage
Nicholas Kan	105923474	Developed and tested the GRU traffic prediction model, assisted with model training, reviewed prediction outputs, route testing, and contributed to report content. Contributed to the ML prediction helper functions section of tbrgs.py by supporting GRU model integration and prediction validation.	25%
Charitha Bandara	104178396	Implemented the data processing component, developed and tested the MLP traffic prediction model, assisted with batch training/evaluation, TBRGS integration, and report preparation. Contributed to the Data loading, ML prediction helper functions, and Main command-line program sections of tbrgs.py.	25%
Xinbo Yu	105961513	Implemented the travel-time conversion component and contributed to the search algorithm integration, including adapting the route-search logic for travel-time edge costs. Contributed to the Edge cost calculation and Top-k route search sections of tbrgs.py, including the integration of the adapted CUS1 / Uniform Cost Search logic. Also assisted with testing and report content.	25%
Santiago Merchant	105342228	Developed and tested the LSTM traffic prediction model, assisted with route guidance testing, SCATS graph preparation, result analysis, and report formatting. Contributed to the Graph construction, ML prediction helper functions, and Output functions sections of tbrgs.py.	25%